SOFTWARE PROJECT LAB / SPL-01

CloneScope

Phase 5: CSV and JSON Report Export

Full Technical Learning Manual

THE MILESTONE

From in-memory analysis results to four durable, inspectable files, without changing the underlying clone or stylometric detection rules.

Prepared for: Raian Huq (BSSE1708)

Institute: Institute of Information Technology (IIT), University of Dhaka

Project: CloneScope - Explainable Code Clone Tagging and Stylometric Analysis

Implementation: C++17, Windows, VS Code, PowerShell

Evidence snapshot: feat/csv-json-report-export at commit 86d7ffc

Manual date: 8 November 2026

Academic-use statement. Clone candidates and coding-style changes are indicators for human review; they are not proof of plagiarism.

Contents

A guided study path from the previous phase to source-code explanation, data formats, export behavior, testing, limitations, and viva preparation.

1. Executive Overview and Scope 3

2. End-to-End Architecture 4

3. Shared Report Data Model 5

4. CSV Serialization in Depth 6

5. JSON Serialization in Depth 9

6. Shared Human-Review Indicators 11

7. Physical File Export and Filesystem Safety 12

8. Integration into application.cpp 14

9. Essential C++17 Syntax and Library Concepts 15

10. Testing Strategy and Verified Evidence 17

11. Practical Demonstration: Windows and PowerShell 19

12. Troubleshooting and Common Errors 21

13. Limitations, Responsible Use and Future Work 21

14. Viva Preparation: Questions and Model Answers 22

15. Source Index and Reproduction Notes 25

RECOMMENDED STUDY ORDER

Read Chapters 1-3 first for the architecture, then 4-8 for exact C++ implementation. Chapters 9-13 support debugging, demonstrations, and viva.

1. Executive Overview and Scope

1.1 What problem this milestone solves

Before Phase 5, CloneScope could print a detailed terminal analysis of a selected directory: recognized source files, function-body clone candidates, aggregate source statistics, naming and commenting deviations, and human-review indicators. That output was useful for a person reading the console but inconvenient to preserve, compare, filter, or import into another program. Long terminal output was especially cumbersome for the controlled 22-student dataset.

Phase 5 adds an export layer. The existing analysis pipeline builds one AnalysisReport object. Independent functions serialize the exact same information to three comma-separated value files and one JSON report. A file-export coordinator creates a fresh output directory for each run. The terminal report continues to work.

1.2 What was already available in Phase 4

- Supported comment removal, tokenization, heuristic function-body recognition, and directory-based source discovery.
- Type-1 body-level candidates and restricted Type-2-only candidates with recorded source locations.
- Aggregate source lines, blank lines, token counts, recognized function-body counts, and student-wise naming/commenting histories.
- An integrated console report with human-review cautions.

1.3 What Phase 5 actually adds

New component	Role	Observable result
csv_export.*	CSV quoting plus three report writers	Three tabular .csv files
json_export.*	JSON string escaping and full nested report	One .json file
report_file_export.*	Directory creation and checked file writes	run_1, run_2, ...
analysis_report.*	Shared human-review reason collection	Consistent console/JSON reasons
application.cpp	Invokes export after analysis	Output folder and filenames in CLI
New smoke tests	Special-character, empty, file and structure checks	Repeatable regression evidence

IMPORTANT SCOPE BOUNDARY

No new Type-3, Type-4, semantic clone analysis, machine-learning classifier, or statistical plagiarism probability is introduced in Phase 5.

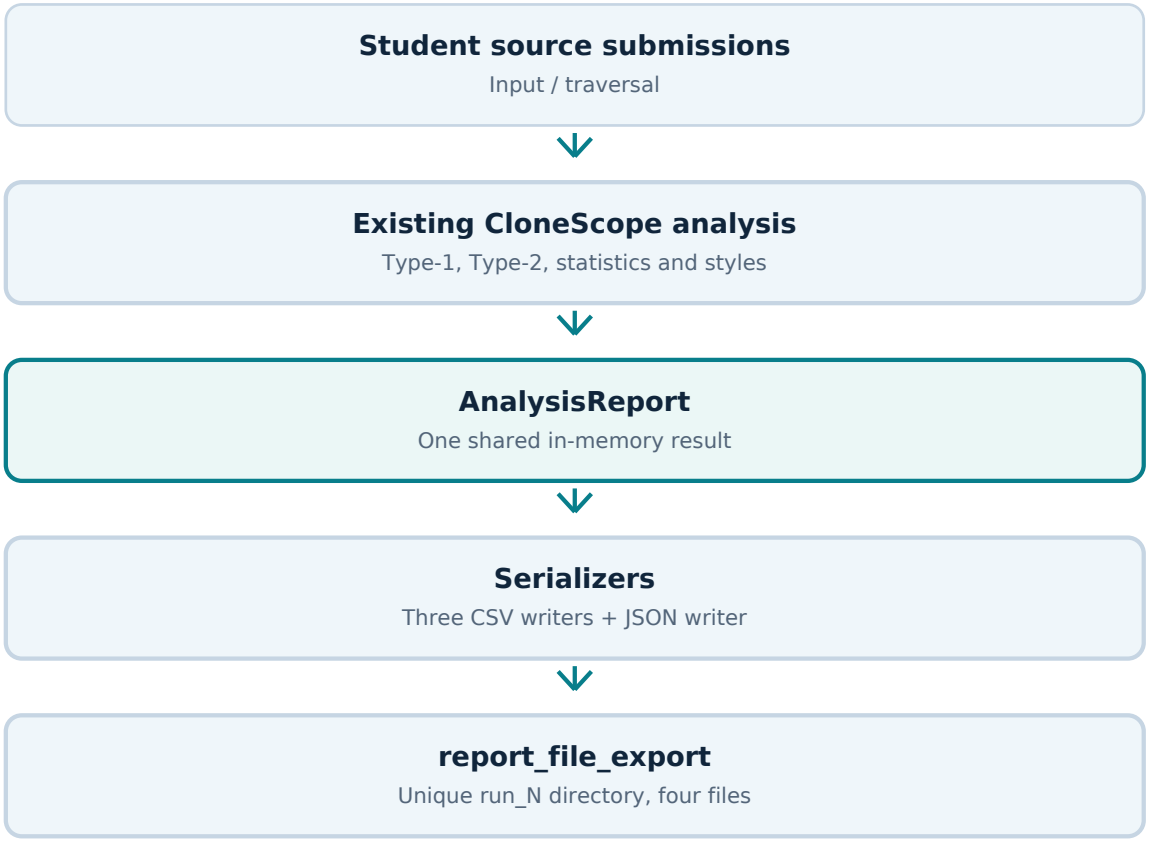
1.4 Definition of completion

The implementation was compiled with C++17 warning flags; all 28 smoke-test executables in the repository were compiled and run with a passing result. A full directory-analysis run on the synthetic 22-student cohort successfully generated four files, and PowerShell parsed both JSON and CSV content. Documentation and the feature-branch merge are separate release-management tasks, not new detection features.

2. End-to-End Architecture

2.1 One source of truth

The central design decision is serialize existing results. Clone detection and historical stylometry are computed by the application pipeline. Phase 5 does not recalculate those findings or silently apply another scoring policy while writing output files. This separation makes it easier to reason about correctness and compare terminal, CSV, and JSON views of the same analysis.



2.2 Responsibilities and dependency direction

Module	Consumes	Produces
application.cpp	Discovered/analyzed files and historical findings	Populated AnalysisReport; export call
analysis_report.hpp	AggregateStatistics, CloneSummaryEntry, StudentStyleFinding	Shared report data model
analysis_report.cpp	AnalysisReport	Console report and HumanReviewIndicators
csv_export.cpp	AnalysisReport	Delimited text written to std::ostream
json_export.cpp	AnalysisReport and shared indicator helper	Nested JSON text written to std::ostream
report_file_export.cpp	Report plus destination root	Four checked physical output files

2.3 Sequence of a directory run

1. The user enters a directory such as `tests/fixtures/cohort22` and chooses recursive scanning.
2. CloneScope discovers supported source files and computes their tokens and source statistics.
3. The existing pipeline compares distinct file pairs and gathers clone candidate details.
4. Historical naming and commenting styles produce one student-finding record per recognized student.
5. The application constructs AnalysisReport once and prints the consolidated terminal report.
6. The exporter creates clonescope_reports/run_N, then writes the three CSV files and JSON file.
7. Success prints the absolute output folder and the four saved filenames; failure returns a nonzero status.

EXECUTION DETAIL
The current export call is in the directory-analysis branch of runCloneScope(). A single-file run still performs its existing per-file analysis but does not invoke this four-file export path.

2.4 Comparing 66 files

Directory analysis counts distinct unordered file pairs. With n successfully analyzed files, the number is n(n - 1) / 2. With 66 files, that is 66 x 65 / 2 = 2,145. This is the number of file pairs compared, not the number of clone candidates. The controlled dataset contains 20 body-level candidate pairs in the recorded report. Confusing these two quantities is a common viva mistake.

3. Shared Report Data Model

3.1 The AnalysisReport structure

Source: src/analysis_report.hpp - simplified formatting

```
struct AnalysisReport {
    std::size_t studentFolderCount = 0;
    std::size_t comparedFilePairs = 0;
    AggregateStatistics aggregate;
    std::vector<CloneSummaryEntry> cloneCandidates;
    std::vector<StudentStyleFinding> studentFindings;
};
```

studentFolderCount is a dataset-level count. comparedFilePairs is the number of examined file pairs. aggregate collects source and candidate totals. cloneCandidates preserves matched pair locations and classifications. studentFindings stores student-wise historical style evaluations. A `std::vector` is an ordered, dynamically sized collection; a `struct` groups related fields.

3.2 StudentStyleFinding

```
struct StudentStyleFinding {
    std::string studentId;
    bool namingEvaluated = false;
    bool namingDeviation = false;
    bool commentingEvaluated = false;
    bool commentingDeviation = false;
};
```

The evaluated flag is crucial. If a student's historical evidence is insufficient or unstable, `namingEvaluated=false` means not evaluable, not 'no deviation'. The serializer retains that distinction using a status label rather than pretending that missing evidence is negative evidence.

3.3 How candidate locations are retained

Each CloneSummaryEntry contains a candidate kind (Type-1 or Type-2-only), the first and second relative source paths, and a starting/ending source-line range for each participant. The CSV writer expands these fields into columns; the JSON writer nests the two locations under first and second. Both formats preserve the pairwise evidence without constructing an inferred all-to-all clone group.

3.4 A report is a snapshot, not an engine

AnalysisReport is an in-memory value object. It does not scan directories, tokenize files, or decide what is plagiarism. This enables test programs to construct small synthetic reports directly, call only the serializer under test, and compare the output to expected strings. That is why the Phase 5 tests are relatively focused.

4. CSV Serialization in Depth

4.1 What CSV means

CSV stands for comma-separated values. Each physical record normally represents a row, and commas separate fields. A first row usually supplies column names. However, a field itself may contain a comma, a quote, a newline, or a carriage return. Without quoting rules, a CSV reader could mistake part of a student ID or path for another column or another record.

4.2 The escapeCsvField() algorithm

The implementation first scans the input for any of four characters: comma, quotation mark, newline, or carriage return. If none appear, the original string is returned. Otherwise it creates a quoted field, writes an extra quotation mark before each quotation mark found inside the field, and closes the quoted field.

Core decision in src/csv_export.cpp

```
bool needsQuotes = false;
for (char ch : field) {
    if (ch == ',' || ch == '"' ||
        ch == '\n' || ch == '\r') {
        needsQuotes = true;
        break;
    }
}
```

Original field	CSV field text	Reason
student01	student01	No quoting required
Hello, world	"Hello, world"	Comma must remain inside one field
He said "review"	"He said ""review"""	Inner quote is doubled
First + newline + Second	"First + newline + Second"	Multiline content is enclosed
empty string	empty string	No unnecessary quotes

4.3 Understanding push_back()

```
std::string result;
result.push_back('"');
for (char ch : field) {
    if (ch == '"') result.push_back('"');
    result.push_back(ch);
}
result.push_back('"');
```

`push_back(ch)` appends one character to the end of an existing `std::string`. When the character is a quotation mark, the algorithm appends it twice so a CSV parser treats the pair as a literal quote inside the field, not the end of that field.

4.4 analysis_overview.csv

The writer first emits the fixed header `metric,value`, then one row for each metric. It reports nine metric names and their calculated values; it does not perform another computation of source statistics.

Verified synthetic cohort result

```
metric,value
student_folders,22
source_files,66
compared_file_pairs,2145
total_source_lines,6973
blank_source_lines,242
total_tokens,30995
recognized_function_bodies,242
type1_candidate_pairs,10
type2_only_candidate_pairs,10
```

4.5 clone_candidates.csv

Each row represents one reported body-level candidate. The 11 columns are type, student_a, assignment_a, path_a, start_line_a, end_line_a, student_b, assignment_b, path_b, start_line_b, and end_line_b. The student ID is derived from the first component of the relative path; the assignment name comes from its filename. Text fields pass through escapeCsvField().

```
// Representative columns, not a complete 11-column row:
type,student_a,assignment_a,path_a,...
Type-1,student01,assignment3.cpp,...
```

The test suite checks candidate classifications, empty lists, and paths that contain special characters. For an empty candidate list the writer still emits the header, producing a valid zero-row data table.

4.6 student_findings.csv

This seven-column report contains student_id, naming_evaluated, naming_deviation, naming_status, commenting_evaluated, commenting_deviation, commenting_status. Evaluation/deviation booleans are written as the literal text `true` or `false`. The status field gives one of three strings: deviation, no_deviation, or not_evaluable.

Evaluated?	Deviation?	Reported status
false	false/ignored	not_evaluable
true	true	deviation
true	false	no_deviation

SECURITY / SCOPE NOTE

CSV quoting protects the structural meaning of commas and quotes. It is not a complete defense against spreadsheet formula interpretation. Future use with untrusted filenames or IDs should consider an explicit spreadsheet-safe output policy.

5. JSON Serialization in Depth

5.1 Why JSON for this report

JSON (JavaScript Object Notation) is a text data format widely read by many languages, including PowerShell. Despite its historical name, it is not limited to JavaScript and CloneScope generates it in C++17. JSON supports nested objects, arrays, quoted strings, Boolean literals, and numbers. Unlike CSV, it naturally represents the fact that each clone candidate contains two locations, and each student may have several review reasons.

5.2 The complete top-level schema

Top-level key	Value type	Meaning
tool	object	Name and format_version
summary	object	Dataset, source and clone totals
clone_candidates	array	Pairwise candidate records
student_findings	array	Historical naming/commenting results
human_review_indicators	object	Distinct students and reasons
interpretation_warning	string	Explicit human-review caution

Structural illustration: ellipses are explanatory placeholders

```
{
  "tool": {"name": "CloneScope", "format_version": 1},
  "summary": { ... },
  "clone_candidates": [ ... ],
  "student_findings": [ ... ],
  "human_review_indicators": { ... },
  "interpretation_warning": "Human review required; ..."
}
```

5.3 How writeJsonString() escapes text

Valid JSON strings require a surrounding pair of double quotation marks. Inside the string, a literal quotation mark, backslash, or control character requires an escaped representation. CloneScope processes the input as `unsigned char`, preventing negative signed-char values from being mistaken for ASCII control characters.

Input character	JSON encoding	Why
double quote	\"	Does not close the JSON string
backslash	\\	Represents a literal backslash
newline	\n	Avoids unescaped line break
carriage return	\r	Control character
tab	\t	Control character

Input character	JSON encoding	Why
backspace / form feed	\b / \f	Control character
other code point below 0x20	\u00XX	Two hex digits encode the byte

Representative part of the control-character fallback

```
constexpr char hex[] = "0123456789ABCDEF";
for (unsigned char ch : value) {
    if (ch < 0x20) {
        output << "\\u00"
               << hex[(ch >> 4) & 0x0F]
               << hex[ch & 0x0F];
    }
}
```

The first nibble is obtained by shifting a byte right four bits; the second uses a bitwise AND with hexadecimal F. Each nibble indexes the 16-character hexadecimal lookup string. The actual implementation also contains `switch` cases for the common special characters and emits ordinary non-control bytes unchanged.

5.4 JSON summary

writeJsonSummary() writes three nested objects. dataset_overview has student folders, source files, and compared file pairs. source_statistics has source-line, blank-line, token and function-body counts. clone_detection contains Type-1, Type-2-only, and total candidate counts. The total is the sum of the two candidate counts.

Cohort22 numerical excerpt

```
"clone_detection": {
  "type1_candidate_pairs": 10,
  "type2_only_candidate_pairs": 10,
  "total_candidate_pairs": 20
}
```

5.5 JSON candidate entries and nested locations

writeJsonCloneCandidates() emits `[]` when there are no candidates. Otherwise, it iterates over the vector by index, writes a comma before every entry except the first, and calls writeJsonLocation() twice for the first and second source locations.

Abbreviated record from structure smoke-test fixture

```
{
  "type": "Type-1",
  "first": {
    "student_id": "student01",
    "assignment": "assignment3.cpp",
    "path": "student01/assignment3.cpp",
    "start_line": 4,
    "end_line": 29
  },
  "second": { ... }
}
```

The JSON string writer is used for every dynamic student ID, path, assignment name, reason, and classification. Numeric line fields are emitted as numbers, not quoted strings. The first path component is interpreted as a student ID under the current directory layout convention.

5.6 JSON student findings

writeJsonStudentFindings() emits an array of records. Each record has a student_id and two nested objects, naming and commenting. Each nested object contains evaluated and deviation as actual JSON Booleans, plus a readable status string. jsonStyleStatus() keeps non-evaluable histories distinct.

Conceptual example matching the tested non-evaluable case

```
{
  "student_id": "student15",
  "naming": {
    "evaluated": false,
    "deviation": false,
    "status": "not_evaluable"
  },
  "commenting": {
    "evaluated": false,
    "deviation": false,
    "status": "not_evaluable"
  }
}
```

5.7 Full-report assembly

writeAnalysisReportJson() writes the opening brace, the tool metadata, then delegates to the summary, clone candidate, student findings, and human-review writers. Finally, it writes the interpretation warning and the closing brace. This is composition: small writers produce logical sections; a larger writer combines them.

Because this prototype constructs JSON text through stream output rather than using an external JSON library, its indentation is not perfectly normalized at every nesting level. This is a presentation issue, not a syntax failure: PowerShell's ConvertFrom-Json successfully parsed the actual generated report in the verified integration run.

VIVA CLARIFICATION
Do not tell an examiner that JSON itself was written in JavaScript. All serializers shown here are C++17 functions. PowerShell parses the generated JSON later to verify the format.

6. Shared Human-Review Indicators

6.1 Why the helper is shared

Human-review indicators are not a third clone-detection algorithm. They summarize reasons already present in clone candidates and student histories. Using one shared collectHumanReviewIndicators() function allows the console report and JSON export to use the same criteria. This avoids two implementations drifting apart when a reason label is changed.

Source: src/analysis_report.hpp

```
using HumanReviewIndicators =
    std::map<std::string, std::set<std::string>>;

HumanReviewIndicators collectHumanReviewIndicators(
    const AnalysisReport &report);
```

6.2 What map and set mean

A `std::map` associates a student ID with a collection of reasons. A `std::set` stores each distinct reason only once. These ordered containers also give predictable iteration order. Multiple clone candidate pairs may involve the same student, but the resulting indicator summary lists the reason once per student.

6.3 Indicator collection algorithm

1. For every clone-candidate entry, map Type-1 to the reason 'Type-1 body-level candidate' and Type-2-only to its corresponding reason.
2. Extract each participant's student ID from the first relative path folder; insert the reason into that student's set.
3. For every student-style finding, add 'Historical naming-style deviation' only if namingEvaluated and namingDeviation are both true.
4. Add 'Historical commenting-style deviation' only if commentingEvaluated and commentingDeviation are both true.
5. Return the map; its number of keys is the number of distinct students who have at least one indicator.

One of the final evidence guards

```
if (finding.namingEvaluated && finding.namingDeviation) {
    indicators[finding.studentId].insert(
        "Historical naming-style deviation");
}
```

6.4 The difference between 20 and 12

The 22-student integration reports 20 body-level candidate pairs (10 Type-1 and 10 Type-2-only). That does not mean 20 distinct students are involved. A student can appear in multiple candidate pairs, and some students also have style-deviation reasons. After deduplication by student ID, the report contains 12 students with at least one human-review indicator. These figures describe different units and are not contradictory.

ETHICAL BOUNDARY
The interpretation warning is mandatory in the report: 'Human review required; similarity is not proof of plagiarism.' An indicator means 'inspect this case with context', not 'the student cheated'.

7. Physical File Export and Filesystem Safety

7.1 The export coordinator's interface

Source: src/report_file_export.hpp

```
bool exportAnalysisReportFiles(
    const AnalysisReport &report,
    const std::filesystem::path &outputRoot,
    std::filesystem::path &createdRunDirectory,
    std::string &errorMessage);
```

The first two parameters are read-only inputs. The third is an output parameter filled with the newly created run path; the fourth receives a diagnostic message if something fails. A true return means all four file writes succeeded. A false return means the caller should not claim that a complete export is available.

7.2 Creating the output root

The function clears its output path and error message, then calls `std::filesystem::create_directories(outputRoot, ec)`. The `std::error_code` overload lets the implementation inspect filesystem errors without relying on that call throwing an exception. If an error occurs, it provides a message and returns false.

7.3 The run_N allocation loop

Condensed logic from src/report_file_export.cpp

```
for (unsigned int number = 1; number <= 10000; ++number) {
    fs::path candidate =
        outputRoot / ("run_" + std::to_string(number));
    ec.clear();
    if (fs::create_directory(candidate, ec)) {
        createdRunDirectory = candidate;
        break;
    }
    if (ec) return false;
}
```

If run_1 already exists, creating it returns false without an error, so the loop tries run_2. The first successfully created candidate is recorded. This preserves earlier output directories. The loop is bounded at 10,000 candidates; if none is available, the function reports failure. Directory creation is the step used to claim a new run name.

7.4 What the path '/' operator means

For `std::filesystem::path`, the slash operator combines path components. It is not numerical division. On Windows, the runtime resolves the platform-appropriate path separators, so source code can use the same C++ expression without manually concatenating backslashes.

7.5 The generic writeFile lambda

To avoid writing the same file-open, write, close, and error-check logic four times, the coordinator declares a lambda with two parameters: a filename and a writer function. Its `auto` parameter permits passing different serializer functions with compatible call shapes. Its `[&]` capture allows the lambda to use variables from the surrounding function by reference.

Simplified excerpt; real code also fills errorMessage

```
auto writeFile = [&](const char *fileName, auto writer) -> bool {
    const fs::path path = createdRunDirectory / fileName;
    std::ofstream file(path);
    if (!file) return false;
    writer(file, report);
    file.close();
    if (!file) return false;
    return true;
};
```

Each serializer accepts an `std::ostream&` and a `const AnalysisReport&`. A `std::ofstream` can be used as an output stream, so each serializer works both with files in production and with `std::ostringstream` in unit tests.

7.6 The four file writes

Order	Filename	Writer
1	analysis_overview.csv	writeAnalysisOverviewCsv
2	clone_candidates.csv	writeCloneCandidatesCsv
3	student_findings.csv	writeStudentFindingsCsv
4	analysis_report.json	writeAnalysisReportJson

After a successful export, a run directory contains exactly the four expected filenames under normal execution. The report_file_export_smoke test verifies successful first and second exports, nonempty files, different directories, and an invalid output location.

7.7 Limits of error handling

The function checks whether files open and whether the stream remains successful after close. On a failure it returns false and gives an explanation. It does not implement transactional rollback: if the third or fourth file fails, a newly created run directory and earlier files may remain. The application reports failure instead of saying that the export completed. This is an honest prototype limitation.

Output paths are relative to the program's current working directory. Running the executable from a different directory may place the report root in a different location. The report directory is excluded by the repository's .gitignore, keeping generated artifacts out of normal commits.

8. Integration into application.cpp

8.1 The point of integration

The application finishes collecting statistics and candidate details before constructing the report. The current source assigns the existing aggregate and vectors to a local AnalysisReport, prints the console report, and then invokes the export coordinator. That ordering means both presentations receive the same computed result, with no second directory traversal or clone comparison.

Actual statements in src/application.cpp

```
AnalysisReport report;
report.studentFolderCount = studentFileIndices.size();
report.comparedFilePairs = filePairCount;
report.aggregate = aggregate;
report.cloneCandidates = candidateDetails;
report.studentFindings = studentFindings;
printCombinedAnalysisReport(cout, report);
```

8.2 Calling the exporter

Directory-analysis branch, src/application.cpp

```
fs::path exportDirectory;
std::string exportError;
if (!exportAnalysisReportFiles(
        report, "clonescope_reports",
        exportDirectory, exportError)) {
    cerr << "Error: CSV/JSON report export failed.\n";
    cerr << exportError << '\n';
    return 1;
}
```

A successful result is followed by a completion banner, the absolute export directory, and four Saved: lines. Failure writes to `std::cerr` and returns 1 from the application function. Returning 0 communicates success to the operating system; returning a nonzero value communicates failure.

8.3 What the user sees

Example from a verified integration run; path abbreviated

```
CSV/JSON REPORT EXPORT COMPLETE
Output folder: C:\...\SPL_01_BSSE1708\clonescope_reports\run_3
Saved: analysis_overview.csv
Saved: clone_candidates.csv
Saved: student_findings.csv
Saved: analysis_report.json
```

8.4 Why this architecture is easier to maintain

The detection logic has one responsibility, reporting has another, and filesystem management has a third. If a future graphical interface is added, it can consume the same JSON structure. If a new CSV column is required, the CSV serializer can be updated without changing the matching rules. If paths or output naming change, most of that work belongs in report_file_export.cpp rather than the algorithms.

9. Essential C++17 Syntax and Library Concepts

9.1 Header files and implementation files

A .hpp file declares the interface other modules may call; the corresponding .cpp file provides implementation. `#pragma once` prevents the compiler from processing the same header repeatedly in one translation unit. An `#include` directive makes declarations visible to a source file.

9.2 References and const

```
void writeAnalysisReportJson(
    std::ostream &output,
    const AnalysisReport &report);
```

The ampersand in `std::ostream&` means a reference to an existing stream. The report argument is const, so the function promises not to modify it through that reference. Passing a large report by const reference avoids an unnecessary deep copy and lets tests use memory-backed streams.

9.3 Output streams

`std::ostream` is a general output stream abstraction. The insertion operator `<<` sends text or numbers to that stream. `std::cout` displays console text; `std::ofstream` writes to a file; `std::ostringstream` collects output into a string for tests.

Stream reuse for a unit test

```
std::ostringstream buffer;
writeJsonSummary(buffer, report);
std::string jsonText = buffer.str();
```

9.4 Vectors, maps and sets

C++ type	Mental model	Actual use
std::vector<T>	Growable ordered sequence	Candidate and finding records
std::map<K,V>	Key-to-value association	studentId to reasons
std::set<T>	Unique ordered values	Distinct reason labels
std::string	Owned character sequence	IDs, paths, serialized fields
std::size_t	Unsigned size/index type	Counts and line indices

9.5 Range-based for loops

```
for (const StudentStyleFinding &finding :
     report.studentFindings) {
    // Read one existing finding at a time.
}
```

The colon means 'for each element in'. The type `const StudentStyleFinding&` binds to each existing vector element without copying it and prevents mutation through that variable.

9.6 Ternary operator

```
const std::string type =
    candidate.kind == CloneCandidateKind::Type1
        ? "Type-1"
        : "Type-2-only";
```

The conditional operator has the form `condition ? value_if_true : value_if_false`. This expression chooses which label belongs to the existing candidate kind.

9.7 Unnamed namespaces

Helper functions such as studentFromRelativePath() and jsonStyleStatus() appear inside an unnamed namespace. This gives them internal linkage: they are implementation details of the current source file, not exported public interfaces. A small helper can still be unit-tested indirectly through its public writer.

9.8 Lambdas, auto and captures

```
auto writeFile = [&](const char *fileName,
                     auto writer) -> bool {
    // Uses createdRunDirectory, report, and errorMessage.
};
```

A lambda is an unnamed callable object. [&] captures needed outer variables by reference. The first parameter is a C-string filename. The second, declared as auto, makes the lambda generic over writer callables. -> bool explicitly states the result type. This is valid modern C++17 syntax, not a regular expression.

9.9 Filesystem aliases and paths

```
namespace fs = std::filesystem;
fs::path outputRoot = "clonescope_reports";
fs::path newFolder = outputRoot / "run_7";
```

The namespace alias shortens repeated use of `std::filesystem`. Path objects handle file/directory components more safely than manually assembled strings. The final string representation depends on the operating system.

9.10 Return values and error messages

`bool` returns only true or false. An additional `std::string&` out-parameter supplies detailed diagnostics. C++ streams convert to a Boolean-like state when checked via `if (!file)`. That test catches failed opening and relevant stream failures after closing.

9.11 C++ compilation and linkage

Each .cpp is compiled as a translation unit. Headers supply declarations; the linker combines object files into one executable. If a test calls a function implemented in another .cpp without linking that file, the compiler might accept the declaration but the linker reports an undefined reference. The comprehensive regression compiled reusable module objects and linked each test against them, excluding the application's main program to avoid duplicate entry points.

10. Testing Strategy and Verified Evidence

10.1 Test layers

The tests deliberately range from small deterministic functions to real filesystem behavior and a controlled end-to-end application run. A single success banner alone is weak evidence; multiple test levels check more independent failure modes.

Layer	Representative tests	Purpose
Scalar escaping	csv_export_smoke; json_string_smoke	Quotes, commas, control characters, empty text
Structured serializers	csv_overview/candidates/student_findings; json_summary/candidates/student_findings	Data columns, arrays, flags, empty collections
Shared logic	review_indicators_smoke	Distinct students, review reasons, non-evaluable cases
Whole JSON shape	json_report_structure_smoke	Tool metadata, nested sections and warning
Filesystem	report_file_export_smoke	Four files, repeat runs, invalid location
Application integration	cohort22 CLI + PowerShell parsing	Real report consistency

10.2 Focused CSV assertions

- A comma in a field triggers quoting; embedded quotes are doubled.
- A newline or carriage return in a field triggers quoting.
- An ordinary or empty field remains correct.
- Overview values are checked against known numerical fixtures and against an empty report.
- Clone candidate records retain class, paired locations and escapable paths; empty vectors preserve headers.
- Student statuses cover deviation, no deviation, not evaluable, and IDs containing a comma.

10.3 Focused JSON assertions

- Plain strings, quotes, backslashes, newlines, control bytes and empty strings.
- Numerical summary values and empty-report behavior.
- Candidate classifications, source paths, line ranges, separators and empty arrays.
- JSON Boolean literals, nested student findings and non-evaluable status.
- Student IDs/paths requiring escaped characters.
- Full report structure with human-review reasons and the warning string.

10.4 Export and regression results

The report_file_export_smoke executable reported PASS for the first export, four nonempty report files, second export, unique run directories, and an invalid output location. In a later comprehensive run, the generated directories included run_4 and run_5, demonstrating the non-overwriting numbering behavior.

The complete smoke-test sweep compiled shared C++ modules, then compiled and executed each test file. The final terminal line was:

```
REGRESSION RESULT: 28/28 TEST EXECUTABLES PASSED
```

The other 17 executables cover earlier pipeline, statistics, clone detection, history and reporting behavior. The comprehensive sweep shows that the export integration did not break the tests exercised by these previous phases.

10.5 Controlled cohort22 results

Verified field	Observed result	How it was checked
Student folders	22	Parsed JSON summary and CSV overview
Source files	66	Parsed JSON summary
Compared file pairs	2,145	Source statistic / known combination
Total source lines	6,973	Fixture expected aggregate
Blank source lines	242	Fixture expected aggregate
Total tokens	30,995	Fixture expected aggregate
Recognized function bodies	242	Fixture expected aggregate
Type-1 candidate pairs	10	Parsed JSON clone_detection
Type-2-only candidate pairs	10	Parsed JSON clone_detection
Candidate records	20	JSON array and CSV row count
Student finding records	22	JSON array and CSV row count
Review-indicator students	12	Parsed JSON indicator student_count

10.6 What the data does and does not prove

The fixture is deliberately synthetic with known expected results. It verifies data flow, serialization, and output consistency under the current implementation. It does not estimate false-positive rates, sensitivity, precision, recall, fairness, or authorship probabilities on real-world submissions. Claiming those quantities would require labeled data and separate evaluation methodology.

11. Practical Demonstration: Windows and PowerShell

11.1 Build and run the complete application

Open PowerShell in the repository root, where src, tests, and samples are present. Use these commands exactly; the `$LASTEXITCODE` value 0 means the last native command completed successfully.

Build CloneScope

```
$sourceFiles = Get-ChildItem .\src -Filter *.cpp |
    Select-Object -ExpandProperty FullName
g++ -std=c++17 -Wall -Wextra -pedantic $sourceFiles -o .\clonescope.exe
$LASTEXITCODE
```

Analyze 22-student controlled cohort

```
@(".\tests\fixtures\cohort22", "y") |
    .\clonescope.exe > .\clonescope_integration.log
$LASTEXITCODE
Get-Content .\clonescope_integration.log -Tail 20
```

11.2 Find the newest run directory

```
$latestRun = Get-ChildItem .\clonescope_reports -Directory |
    Where-Object { $_.Name -match '^run_\d+$' } |
    Sort-Object { [int]($_.Name -replace '^run_', '') } -Descending |
    Select-Object -First 1
$latestRun.FullName
```

The expression matches only folder names like run_1, run_10 and so on. It converts the suffix to a number before sorting, avoiding lexicographic ordering that would incorrectly place run_9 after run_10.

11.3 Open the generated files directly in VS Code

```
code -r (Join-Path $latestRun.FullName "analysis_overview.csv")
code -r (Join-Path $latestRun.FullName "clone_candidates.csv")
code -r (Join-Path $latestRun.FullName "student_findings.csv")
code -r (Join-Path $latestRun.FullName "analysis_report.json")
```

The optional Rainbow CSV extension can color CSV columns; it is a VS Code viewing aid, not a runtime dependency of CloneScope.

11.4 Parse JSON and import CSV

```
$report = Get-Content -Raw (
    Join-Path $latestRun.FullName "analysis_report.json") |
    ConvertFrom-Json -ErrorAction Stop
$report.summary.dataset_overview | Format-List
$report.summary.clone_detection | Format-List
$report.human_review_indicators.student_count
```

```
Import-Csv (Join-Path $latestRun.FullName "analysis_overview.csv") |
    Format-Table -AutoSize
(Import-Csv (Join-Path $latestRun.FullName "clone_candidates.csv")).Count
(Import-Csv (Join-Path $latestRun.FullName "student_findings.csv")).Count
```

These checks are an important integration demonstration: the source application writes text, a separate tool parses that text, and the parsed values match the known controlled dataset. This goes beyond merely checking that a file exists.

12. Troubleshooting and Common Errors

Symptom	Likely explanation	Safe next step
Compilation says 'AnalysisReport' unknown	Header declarations are missing or include order is incorrect	Verify analysis_report.hpp contains the struct before functions using it
Undefined reference at link time	Required .cpp object not linked	Build all src/*.cpp for application, or add dependencies to the test link
Compiler exits with code 0 but old report appears	You inspected an older run_N folder	Pick the highest numeric run suffix
ConvertFrom-Json fails	Invalid JSON or incomplete file	Read raw file, check for truncated output and export errors
CSV row has too many columns	A field was not quoted/escaped	Check escapeCsvField for commas, quotes and newline handling
No CSV files after single-file run	Export is on directory-analysis branch	Choose a source directory to run complete cohort analysis
File export returns false	Directory or file could not be created/written	Read errorMessage and check permissions/location
Git displays generated reports as untracked	Ignore rule missing or file moved outside ignored folder	Verify clonescope_reports/ in .gitignore

12.1 The interrupted compile confusion

If a native compilation command is cancelled with Ctrl+C, a later `$LASTEXITCODE` does not retroactively prove that interrupted command completed. Compile again and check its exit code immediately. Likewise, a successful JSON parsing command must receive actual JSON text in the variable; an empty or stale variable is not a valid integration check.

12.2 Red terminal text and VS Code suggestions

PowerShell prints errors in red, including attempts to create an already-existing file with New-Item. That does not mean the rest of the repository is corrupted. VS Code may also recommend a CSV viewer extension. It is optional and changes presentation, not the report files generated by C++.

13. Limitations, Responsible Use and Future Work

13.1 Current implementation limitations

- Function-body recognition is heuristic and is not a full C/C++ parser.
- The current Type-2 comparison supports consistent identifier renaming; it is not a general semantic equivalence detector.
- CSV/JSON exports repeat precomputed results and do not validate correctness of the detection rules themselves.
- Current JSON output is structurally valid in tested cases but not perfectly indented at every nested level.

- The exporter does not roll back an incomplete run directory after a later file-write failure.
- CSV quoting is not equivalent to spreadsheet-formula neutralization for untrusted content.
- The built-in report formats have a JSON format_version but no comprehensive long-term schema migration system.
- Cohort22 proves test-fixture behavior; performance at large scale and real-data accuracy remain unmeasured.

13.2 Ethical boundaries

Similarity may arise from common algorithms, assignment templates, short solutions, or independently developed work. Changes in naming style or comment density can arise from legitimate learning, constraints, collaboration rules, or personal circumstances. A responsible examiner must inspect code context and assignment policy. CloneScope's output is a set of review signals, not an automated misconduct verdict.

13.3 After-midterm roadmap

Potential future work includes restricted Type-3 clone detection and carefully scoped Type-4 candidate screening, broader C/C++ syntax coverage, stronger input validation, more scalable candidate search, structured export schema documentation, and performance evaluation. Any evaluation on authentic student submissions requires appropriate permissions, anonymization and documented limitations.

13.4 Release and documentation checklist

- Preserve all Phase 1-4 documentation; add this Phase 5 manual without replacing historical material.
- Update README status, source-component listing, output workflow and future-work limitations to reflect completed Phase 5 code.
- Run git diff --check and inspect exactly which files are staged.
- Commit only intended documentation changes and push the feature branch.
- Open/review the feature-to-main Pull Request; check the changed-file list and tests before merging.
- After merge, fetch and synchronize local main; verify status and latest commit.

14. Viva Preparation: Questions and Model Answers

Q01. What is the objective of Phase 5?

To export already-computed CloneScope results into reusable CSV and JSON reports, with separate output directories per run.

Q02. Did Phase 5 add Type-3 or Type-4 detection?

No. Type-1, restricted Type-2-only and stylometric analysis came from earlier phases. Type-3/Type-4 remain future work.

Q03. Why keep AnalysisReport as a shared model?

It centralizes completed results so console, CSV and JSON views do not recompute or disagree about detection.

Q04. What does escapeCsvField do?

It surrounds fields containing comma, quote, newline or carriage return with quotes, and doubles inner quotation marks.

Q05. What if a CSV candidate list is empty?

The CSV serializer outputs the column header but zero data rows.

Q06. Why generate JSON as well as CSV?

JSON represents nested candidate locations and per-student review reasons naturally; CSV is convenient for flat spreadsheet-style analysis.

Q07. Did you write JSON in JavaScript?

No. src/json_export.cpp generates text using C++17 output streams. PowerShell later parses it for validation.

Q08. How does JSON encode a newline inside a string?

The JSON string writer emits a backslash-n escape rather than an unescaped physical newline.

Q09. What does \u00XX represent?

An escaped JSON representation of a control byte below hexadecimal 20 using two hex digits.

Q10. Why is unsigned char used?

It keeps byte values non-negative during control-byte checks, avoiding signed-char surprises.

Q11. Why are evaluated and deviation separate flags?

A lack of sufficient history must be distinguished from an evaluated history with no deviation.

Q12. What is the meaning of not_evaluable?

There is insufficient, ambiguous, or unstable historical evidence; it is not a clean result.

Q13. How are review indicators collected?

The helper adds pairwise clone reasons and evaluated style deviations into a map from student ID to a set of reasons.

Q14. Why map and set?

Map groups by student ID; set deduplicates each reason and keeps a consistent order.

Q15. Why is there a difference between 20 candidates and 12 review students?

The first counts body-level candidate pairs; the second counts distinct student IDs with at least one reason.

Q16. How are student IDs derived from clone paths?

From the first relative directory component; the assignment name comes from the filename.

Q17. What is std::ostream?

A general C++ output-stream interface, usable with console, file, or string-backed streams.

Q18. Why pass AnalysisReport by const reference?

It avoids copying a large structure and protects it from accidental modification by reporting functions.

Q19. What does [&] mean in the file-writing lambda?

The lambda captures needed variables from the surrounding function by reference.

Q20. What does auto writer mean?

A generic lambda parameter that accepts compatible serializer callables without spelling out their function-pointer type.

Q21. Why return bool and also take errorMessage?

The Boolean states success/failure; the string tells the caller what operation failed.

Q22. Why are outputs placed in run_N folders?

Each successful new directory name preserves previous runs and avoids overwriting their four report files.

Q23. How does std::filesystem::path / work?

It appends path components; it is not arithmetic division.

Q24. Does a partial export automatically roll back?

No. The function reports failure, but some files in that new run directory may remain.

Q25. What does the 28/28 result establish?

All 28 repository smoke-test executables compiled and passed their designed checks in the recorded regression.

Q26. What does 2,145 refer to?

The number of distinct unordered file pairs among 66 analyzed files: 66 x 65 / 2.

Q27. Are these counts proof of plagiarism?

No. They are synthetic-fixture indicators intended for contextual human review.

Q28. Why doesn't the single-file run export all four reports?

The present export integration is inside the directory-analysis path, which computes the cross-file dataset report.

Q29. How can an examiner validate JSON independently?

Parse analysis_report.json with PowerShell ConvertFrom-Json and inspect the nested fields.

Q30. How do you explain your project in 30 seconds?

CloneScope analyzes supported C/C++ function bodies for Type-1 and restricted Type-2-only similarity, combines source and historical style evidence, reports explainable review indicators, and exports the results to CSV and JSON for inspection.

15. Source Index and Reproduction Notes

15.1 Primary implementation references

Repository file	What to read and explain
src/analysis_report.hpp	AnalysisReport, StudentStyleFinding and HumanReviewIndicators
src/analysis_report.cpp	Shared reasons and consolidated console report
src/csv_export.hpp/.cpp	CSV escaping and three CSV writers
src/json_export.hpp/.cpp	JSON escaping and full report composition
src/report_file_export.hpp/.cpp	Run directory allocation, writer lambda and I/O checks
src/application.cpp	Construction of AnalysisReport and export invocation
tests/*_smoke.cpp	Deterministic coverage and regression checks
tests/fixtures/cohort22/	Controlled 22-student input and expected outcomes

15.2 Repository and evidence provenance

The implementation descriptions in this manual were checked against the public GitHub feature branch at commit 86d7ffc: https://github.com/raian-x8/SPL_01_BSSE1708. The recorded run/test outcomes are from the developer's Windows PowerShell terminal evidence. This manual itself is a learning and review artifact, not a claim that its PDF was part of that earlier commit.

The test file count (28), cohort fixture counts, and export writer names refer specifically to the feature snapshot documented above. Future commits may change the code or schema, so an examiner should compare against the checked-out commit or the final merged main revision.

15.3 Suggested live demo order

1. Show the repository branch, last meaningful commits, and the 28/28 regression line.
2. Build the full C++17 application and run the synthetic cohort22 directory analysis.
3. Point to terminal clone candidate and style findings, including the warning.
4. Open the new report run_N directory and show its four files.
5. Open analysis_overview.csv, clone_candidates.csv and student_findings.csv in that order.
6. Open analysis_report.json, identify the nested schema, and parse it with PowerShell.
7. Explain the source-level functions that produced the results, and finish with current limits.

ONE-MINUTE SUMMARY

Strong viva answer: 'The analysis engine was built in prior phases. In Phase 5 I designed an output pipeline around one AnalysisReport, wrote CSV and JSON serializers, integrated checked file generation, and verified the results with 28 passing smoke-test executables and a controlled 22-student run.'